

Regular Expression Matching

A dynamic programming approach

Supporting `.` (any single character) and `*` (zero or more of the preceding element)

1 The problem

Given a string s and a pattern p , decide whether p matches *all* of s . The pattern language has exactly two metacharacters:

- `.` matches any single character;
- `*` matches zero or more copies of *the element immediately before it*.

The second rule is the whole difficulty. A `*` is never a token on its own — it always forms a pair with the character to its left, and that pair may consume $0, 1, 2, \dots$ characters of s . Since the right choice is not knowable locally, a greedy left-to-right scan cannot work. For instance `"aaa"` against `"a*a"` matches only if `a*` takes two of the three `a`'s and *gives the last one back* to the literal `a`.

2 The state

Let $m = |s|$ and $n = |p|$, and define a table of booleans of size $(m + 1) \times (n + 1)$:

$dp[i][j]$ = “the first i characters of s are matched by the first j characters of p ”.

The $+1$ in each dimension is essential: the empty prefix is a genuine state, and most of the interesting base cases live in row 0. The answer to the problem is $dp[m][n]$.

Note the index convention that follows from this: cell $[i][j]$ talks about the characters $s[i - 1]$ and $p[j - 1]$, one-based in the table and zero-based in the strings.

3 Base cases

$dp[0][0] = \text{true}$. The empty pattern matches the empty string.

$dp[i][0] = \text{false}$ for $i > 0$. A pattern with nothing left in it cannot consume any character.

Row 0 is the subtle one. A *non-empty* pattern can still match the empty string, but only by way of `x*` pairs that each choose zero occurrences:

$$dp[0][j] = \begin{cases} dp[0][j - 2] & \text{if } p[j - 1] = * \\ \text{false} & \text{otherwise.} \end{cases}$$

Skipping this row is the classic bug: it is what makes `"aab"` against `"c*a*b"` fail, because the leading `c*` must be allowed to vanish.

4 The transition

Everything is decided by looking at the last pattern character, $p[j - 1]$.

Case 1 — an ordinary character or .

The pattern character must consume exactly one character of s , so both strings shrink by one:

$$\text{dp}[i][j] = \text{dp}[i-1][j-1] \wedge (p[j-1] = . \vee p[j-1] = s[i-1]).$$

Case 2 — *

The star pairs with $p[j-2]$. Two independent readings, combined with **or**:

$$\text{dp}[i][j] = \underbrace{\text{dp}[i][j-2]}_{\text{zero occurrences}} \vee \underbrace{(\text{dp}[i-1][j] \wedge (p[j-2] = . \vee p[j-2] = s[i-1]))}_{\text{one more occurrence}}$$

- *Zero occurrences* throws the whole **x*** pair away — hence the jump of **two** columns, $j-2$, not one. The string index i is untouched, because nothing was consumed.
- *One more occurrence* consumes a single character of s (so $i-1$) while *staying on the same star* (so j , unchanged). This is the cell directly above.

5 Why “zero or more” needs no loop

A natural objection to the rule above: the star may swallow any number of characters, so surely we must try every count $k = 0, 1, 2, \dots$ and scan the rest of the string?

We do — but the scan is already inside the cell above us. Write m_i for the predicate “ $s[i-1]$ matches the starred element”. Unrolling the recurrence for Case 2 gives

$$\begin{aligned} \text{dp}[i][j] &= \text{dp}[i][j-2] && (\text{star eats } 0) \\ &\vee m_i \wedge \text{dp}[i-1][j-2] && (\text{star eats } 1) \\ &\vee m_i \wedge m_{i-1} \wedge \text{dp}[i-2][j-2] && (\text{star eats } 2) \\ &\vee m_i \wedge m_{i-1} \wedge m_{i-2} \wedge \text{dp}[i-3][j-2] && (\text{star eats } 3) \\ &\vee \dots \end{aligned}$$

which is exactly the exhaustive scan over every possible repetition count. The point of the table is that we never write that loop: $\text{dp}[i-1][j]$ already summarises the entire tail of the disjunction, computed one row earlier. Each cell therefore costs $O(1)$ instead of $O(m)$.

This is also where the exponential blow-up of naive backtracking disappears. “Give a character back” is not an undo operation — it is simply the other branch of an **or** whose value has already been computed and stored.

6 A worked example

Take $s = \text{"aab"}$ and $p = \text{"c*a*b"}$. Rows are prefixes of s , columns are prefixes of p ; ε denotes the empty prefix.

	ε	c	*	a	*	b
ε	T	F	T	F	T	F
a	F	F	F	T	T	F
a	F	F	F	F	T	F
b	F	F	F	F	F	T

Reading the interesting cells:

- $dp[0][2] = T$: the pair $c*$ chooses zero occurrences, so an empty pattern prefix remains and $dp[0][0]$ is inherited.
- $dp[1][4] = T$: $a*$ takes one a . Its neighbour $dp[2][4] = T$ takes two — the same star, one row further down.
- $dp[3][5] = T$: the literal b matches $s[2]$, inheriting from $dp[2][4]$ diagonally. That cell is the answer.
- Column c is entirely false, as c occurs nowhere in s ; the pattern survives only because the star lets the pair vanish.

The backtracking case $s = "aaa"$, $p = "a*a"$ is worth tracing too:

	ϵ	a	*	a
ϵ	T	F	T	F
a	F	T	T	T
a	F	F	T	T
a	F	F	T	T

The final column is the literal a . Each of its true cells inherits diagonally from the $*$ column one row up — i.e. the star stopped one character early, of its own accord. No backtracking machinery was needed.

7 A better shape: tokenise the pattern first

Everything above works on the raw characters of p , and pays for it in two places: the inner loop must *look ahead* to discover whether the next character is a $*$, and the zero-occurrence branch has to jump $j - 2$ columns. Both awkwardnesses come from the same source — a $*$ is not a thing in its own right, it is a *quantifier attached to the element before it*.

So parse the pattern once, into a list of tokens:

```
enum Elem { Char(u8), Dot }           // what can match one character
enum Tok  { One(Elem), Star(Elem) }   // ...and how many times
```

The parse is a single left-to-right pass: read an element, peek at the next byte, and if it is $*$ consume that too and emit `Star` instead of `One`. It is $O(n)$ and, given the problem's guarantee that every $*$ has a preceding element, it cannot fail.

Note that $*$ is deliberately *not* a third variant of `Tok`. Making it one would leave invalid patterns representable — a leading star, or two in a row — and every later stage would have to defend against states the parser should have ruled out.

The recurrence, restated

Let t be the token list. **The second dimension of the table is now the number of tokens, not the number of pattern characters** — this is the substantive change, and it is worth stating plainly. Write

$$n' = |t| = n - \#\{* \text{ in } p\},$$

since every star merges into the element on its left instead of occupying a column of its own. For $p = "c*a*b"$ that is $n = 5$ pattern characters but only $n' = 3$ tokens: $c*$, $a*$, b .

The table is therefore $(m+1) \times (n'+1)$, and each column is one token rather than one pattern character. Cell $[i][j]$ now reads: “the first i characters of s are matched by the first j *tokens* of the pattern.”

$$\text{dp}[i][j] = \begin{cases} \text{dp}[i-1][j-1] \wedge e.\text{matches}(s[i-1]) & \text{if } t[j-1] = \text{One}(e) \\ \text{dp}[i][j-1] \vee (\text{dp}[i-1][j] \wedge e.\text{matches}(s[i-1])) & \text{if } t[j-1] = \text{Star}(e) \end{cases}$$

with $\text{dp}[0][0] = \text{true}$ and row 0 collapsing to $\text{dp}[0][j] = \text{star}(t[j-1]) \wedge \text{dp}[0][j-1]$.

The improvement is not cosmetic:

- the zero-occurrence case is now $j-1$, not $j-2$ — skipping a token *is* one step back — which removes the `usize` underflow hazard entirely;
- the lookahead disappears from the inner loop; the token already knows whether it is starred;
- row 0 becomes a one-liner;
- the two cases are variants of an `enum`, so `match` makes them exhaustive and the compiler checks that nothing was forgotten.

The same examples, over tokens

$s = \text{"aab"}, p = \text{"c*a*b"}$, which tokenises to $[\text{c*}, \text{a*}, \text{b}]$ — five columns become three:

	ε	c*	a*	b
ε	T	T	T	F
a	F	F	T	F
a	F	F	T	F
b	F	F	F	T

and $s = \text{"aaa"}, p = \text{"a*a"}$, tokenising to $[\text{a*}, \text{a}]$:

	ε	a*	a
ε	T	T	F
a	F	T	T
a	F	T	T
a	F	T	T

Same answers, narrower tables, and no column ever holds a bare `*`. Row 0 is now readable at a glance: it stays true exactly as long as the tokens keep being starred.

8 Complexity

Every cell is filled in constant time, so the character-level table runs in $O(mn)$ time and $O(mn)$ space. With the problem's limits of $m, n \leq 20$ that is at most $21 \times 21 = 441$ cells.

Over tokens the bound is $O(mn')$, with $n' \leq n$: tokenising adds one $O(n)$ pass up front and then shrinks every dimension that follows, because each `x*` pair costs one column instead of two. Same asymptotic class, strictly fewer cells — for $p = \text{"c*a*b"}$, 4×4 instead of 4×6 .

Space can be reduced to $O(n')$ by keeping only the previous row, since the `One` case reads row $i-1$ and the `Star` case reads row $i-1$ and row i . It is rarely worth the loss of clarity at this size.

For comparison, a recursive matcher without memoisation is exponential. The pattern

a*a*a*a*a*a*a*a*a*b

against twenty `a`'s explores an enormous number of ways to split the string among the nine stars before concluding that the trailing `b` can never match.

9 Notes for a Rust implementation

- Work over `s.as_bytes()` and `p.as_bytes()` and compare `u8` values against `b'*` and `b'.'`. The input is ASCII, and byte slices give $O(1)$ indexing that `String` deliberately does not.
- Give `Elem` the matching rule as an inherent method, so the DP reads as prose:

```
impl Elem {
    fn matches(&self, c: u8) -> bool {
        match self { Elem::Dot => true, Elem::Char(x) => *x == c }
    }
}
```

- `vec![vec![false; n + 1]; m + 1]` is the natural table. A flat `Vec<bool>` indexed by `i * (n + 1) + j` avoids the pointer indirection if you prefer.
- Indices are `usize`, so a subtraction that goes below zero *panics* rather than wrapping to a negative number. Driving the loops as `for i in 0..=m / for j in 1..=n` keeps every $j - 1$ well defined — and with tokens there is no $j - 2$ left to worry about.
- Fill order matters: row 0 first (left to right), then rows 1 through m . The `Star` case reads `dp[i][j - 1]` from the current row, which must already be filled to its left.
- Matching on a pair of tokens is where Rust's `match` earns its keep: write `match &tokens[j - 1] { Tok::One(e) => ..., Tok::Star(e) => ... }` and the compiler guarantees both cases are handled.

All four example tables in this document were produced by running the recurrence, not by hand.